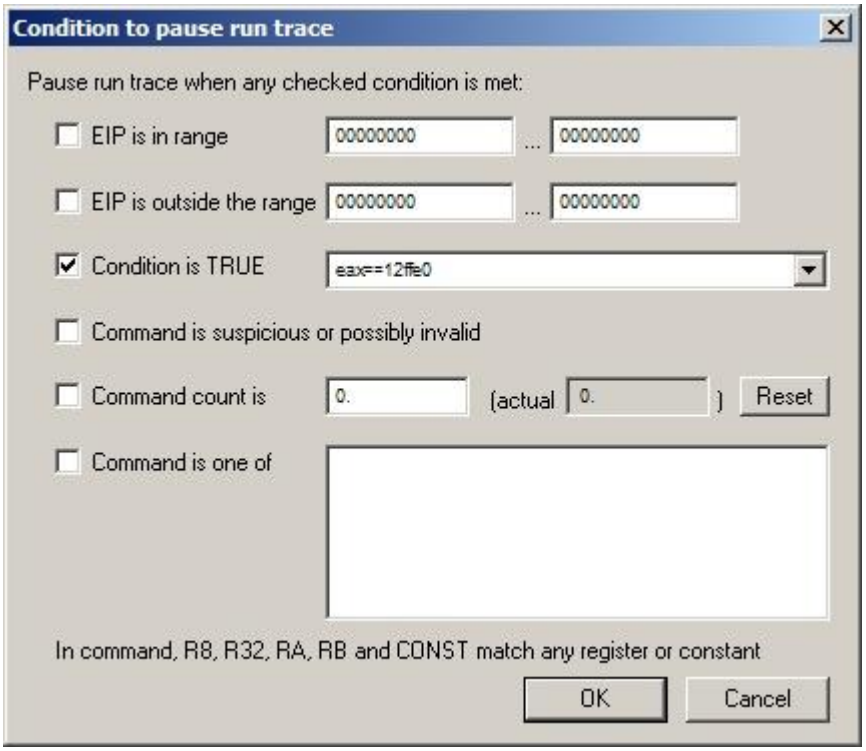
Programa:	AWMAWorkshop 1.76	
Protección:	Empaquetado con ASProtect y limitaciones varias	
Objetivos:	Desempaquetarlo y evitar todas las restricciones	
URL descarga:	---	
Dificultad:	Los ASProtect cuestan al pricipio pero luego…	
Herramientas:	OllyDbg, ImpRec, IDA Pro y UltraEdit32	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 22-04-2005

Packer: ASProtect 1.22 - 1.23 Beta 21
Compilado en C++ 6.0 con msvcrt

Yo siempre me hago un cuadro-resumen de todos los datos importantes que voy a necesitar para el desempaquetado final del ejecutable, doy por hecho que los que se quieran enfrentar a un ASProtect sabe lo que es una excepcion y todo eso, si no no problema que vaya a la lista CrackslatinoS y alli encontrara infinitos tutos acerca de este compresor y no es broma los hay realmente buenos, asi que no me centro en profundizar en estos aspectos que como digo estan muy bien explicados, pienso ademas que todos conocemos el metodo de las excepciones de Ricardo Narvaja para encontrar el OEP asi que vamos a por faena:

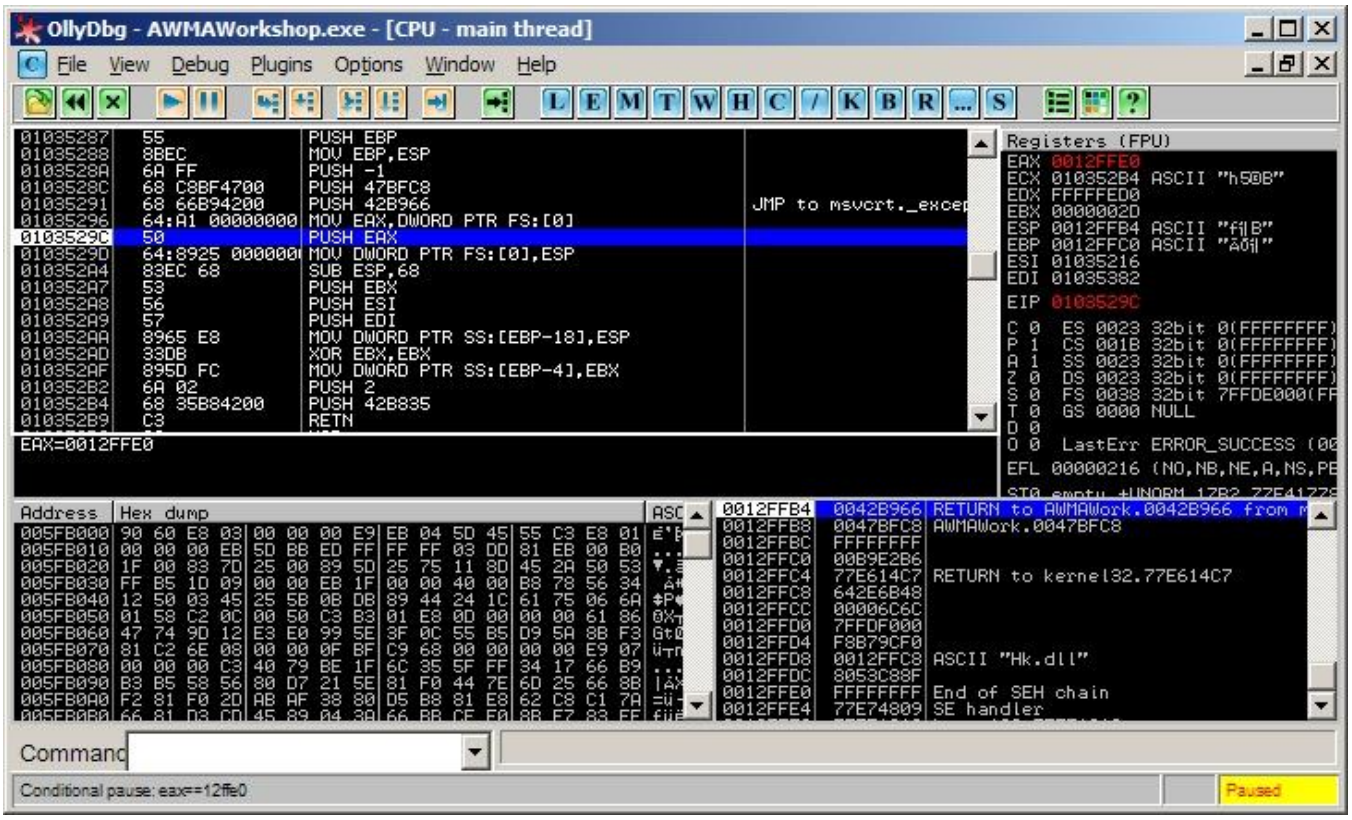
Ultima excepcion :1022d51
OEP: 42b835 (hay SB => eax=12ffe0)
IAT => Inicio: 477000
Final: 477ce8
Largo: ce8
Call semi-mágico: 1022671

Hago una traza a partir de la última excepción para pillar los Stolen Bytes que ejecutó el packer en su propia rutina para despistarnos mas, estando en esta última excepción ponemos un 'BP on Memory Acces' en la sección de ASProtect de la que venimos y que podemos ver en el stack sabemos que es esta porque alli tiene instalado el manejador de excepciones (SE handler) y que podremos ver al lado, al caer aquí traceamos un poco pasamos el primer call con Trace Over (f8) y a partir de aquí lanzamos la traza, ojo que si pasamos sobre el call que hay en :1035f1d irá cambiando variara) y sacará el programa (además en vuestra máquina esta dirección puede seguramente variar), desde aquí ahora vamos a "Debug -> Open or clear run trace" luego a set condition y lo configuramos así:



Ahora pulsamos en el botoncillo que tiene unos puntos suspensivos y desde aquí con el botón derecho elegimos "Log to file" para guardar la traza (esto es importante pues ASProtect borrará sus huellas y asi guardaremos los

lugares por donde pasó, jeje) elegimos una ubicación para el archivo y listos con Ctrl+F11 comenzamos un “Trace Into” para que entre en los calls/subcalls, etc… esto puede tardar dependiendo de la potencia de la máquina, de vez en cuando ir mirando que no estemos en un bucle, si pasa esto vamos a “Debug -> Close run trace” salimos del bucle manualmente con BPX al final y f9 quitamos el BPX y de nuevo Ctrl+f11 para seguir con la traza, para verla vamos al botón de los puntos suspensivos también, desde allí veremos por donde va pasando y si estamos en un bucle ok? Espero que nadie se me pierda, vamos… iniciamos la traza y si vamos mirando veremos que esta entrando en bucles tontos que estan para molestar precisamente así que el proceso a seguir es el indicado antes, o sea, cerramos la traza salimos del bucle con un BPX a la salida luego quitamos el BPX por si las moscas (chequeo CRC de ASProtetc) y continuamos de nuevo con la rtaza con Ctrl+f11, yo aconsejo que cada vez que hagamos esto picar en el botón de los puntos suspensivos y con el botón derecho del ratón ya dentro seleccionar “Close file” luego ir y eliminar el archivo pues no habrá más que basura y elegir de nuevo “Log to file” de esta manera seremos mucho más selectivos con los contenidos del archivo dejando la basura al margen, jeje ya veréis que el archivo resultante no es muy grande, esto lo digo como opción personal porque yo lo hago así aunque lógicamente no es indispensable se puede obviar y hacerlo como quiera cada uno, y dicho esto seguimos a lo nuestro, el proceso de cerrar la traza se repetirá almenos unas tres o cuatro veces pero nada serguimos… al poquitín Olly detendrá la ejecución muy obediente porque eax ya tiene su valor “12ffe0” veamos que hay:



El valor de eax no se puede apreciar muy bien porque no quise hacer una instantánea muy grande pero vale exactamente 12ffe0 como sabemos es el valor que tenía en el OEP pero si nos fijamos bien vemos que estamos justo en la ejecución de los Stolen Bytes…!!! Son estos:

Push ebp
Mov ebp, esp
Push -1
Push 47bfc8
Push 42b966
Mov eax, dword ptr fs:[0]
Push eax
Mov dword ptr fs:[0], esp
Sub esp, 68
Push ebx
Push esi
Push edi
Mov dword ptr [ebp-18], esp
Xor ebx, ebx
Mov dword ptr [ebp-4], ebx
Push 2

Despues ya viene el push del falso OEP y el ret que nos lleva a él que no forman parte de los stolen claro, asi que vamos a arreglar la IAT con ImpRec, le damos los siguientes datos:

OEP: 2b835
IATrva: 77000
IATlenght: ce8

Pulsamos sobre “Get Imports” y saldrán tres entradas inválidas, éstas son las que no puedo reparar el call semi.-mágico per no importa porque podemos arreglarlas nosortos mismos incluso con el plugin de ASProtect 1.22 que trae por defecto ya las soluciona aunque personalmente prefiero verificar las entradas porque aunque pocas he detectado que puede equivocarse, hago una pequeña tabla con las entradas inválidas:

IATrva:	Apunta a:	API correcta:
7709c	10213c0	GetCurrentProcessId
770d8	1021360	GetModuleHandleA
7711c	1020ef0	GetProcAddress

Os muestro las entradas a continuación:

Entrada 1:

```
DISASSEMBLER / HEX VIEWER
010213C0    push ebp
010213C1    mov ebp,esp
010213C3    call 01015170     // = kernel32.dll/01D4/GetVersion
010213C8    mov eax,[1026CF4]     // DWORD value: 00000154
010213CD    pop ebp
010213CE    retn
```

claramente correcta, esta deberá apuntar a GetCurrentProcessId, mirad que retorna en eax un valor tres digitos que es el PID del programa, este valos podria haber sido de solo 2 digitos tambien pero no cambiaria nada las cosas.

Entrada 2:

```
DISASSEMBLER / HEX VIEWER
01021360    push ebp
01021361    mov ebp,esp
01021363    mov eax,[ebp+8]
01021366    test eax,eax
01021368    jnz short 0102137D
0102136A    cmp dword ptr [1026978],400000     // ~= msvcirt.dll/0196/?sunk_wit
01021374    jnz short 0102137D
01021376    mov eax,[1026978]     // ~= msvcirt.dll/0196/?sunk_with_stdio@ios@@
0102137B    jmp short 01021383
0102137D    push eax
0102137E    call 01015158     // = kernel32.dll/016F/GetModuleHandleA
01021383    pop ebp
01021384    retn 4
```

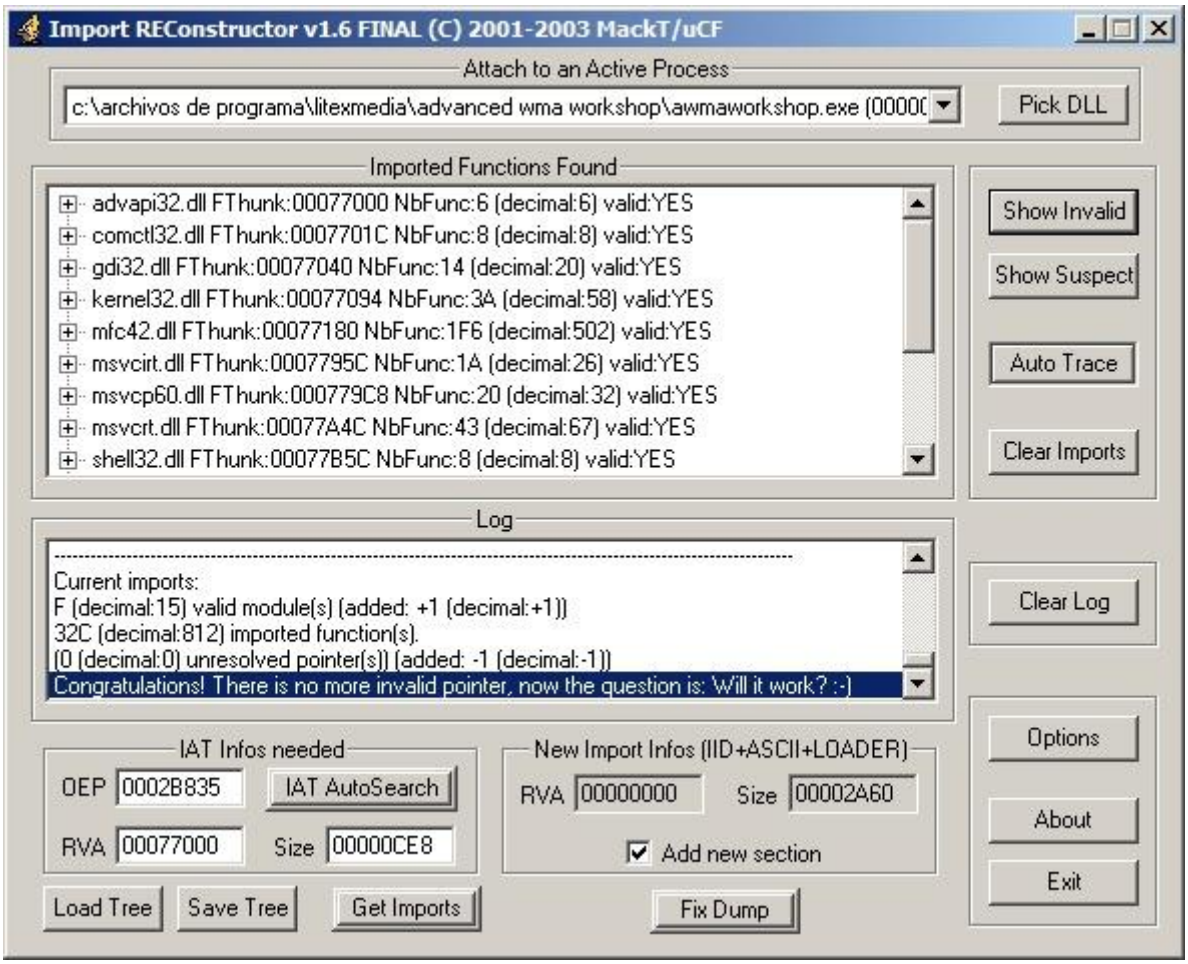
Sin comentarios, tambien correcta y a GetModuleHandleA

Entrada 3:

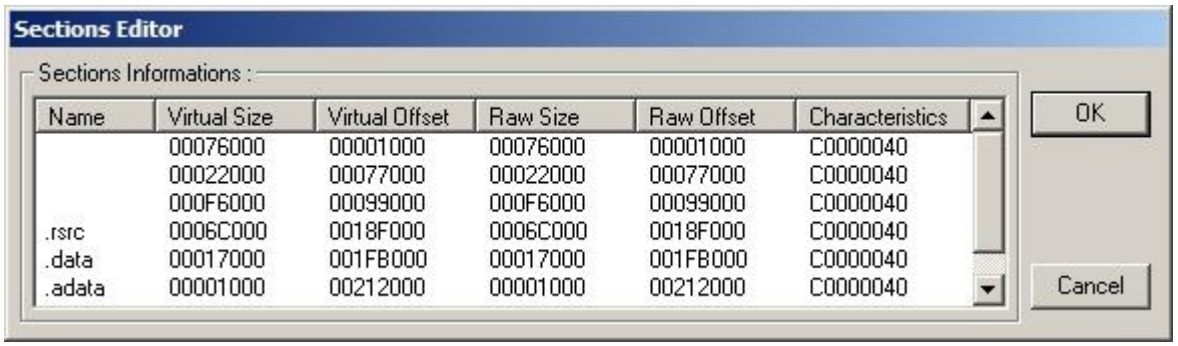
```
DISASSEMBLER / HEX VIEWER

01020EF0    push ebp
01020EF1    mov ebp,esp
01020EF3    mov edx,[ebp+C]
01020EF6    mov eax,[ebp+8]
01020EF9    mov ecx,[102543C]     // DWORD value: 01025314
01020EFF    mov ecx,[ecx]
01020F01    cmp ecx,eax
01020F03    jnz short 01020F0E
01020F05    mov eax,[edx*4+1025350]     // DWORD value: 00000000
01020F0C    jmp short 01020F15
01020F0E    push edx
01020F0F    push eax
01020F10    call 01015160     // = kernel32.dll/0191/GetProcAddress
01020F15    pop ebp
01020F16    retn 8
```

Al igual que las dos anteriores esta es también claramente correcta y es GetProcAddress como se puede observar. Ahora que tenemos las entradas resueltas ImpRec tendrá este aspecto:



Vemos que dice “Valid: YES” en todos los Thunks, ahora pulsamos “Fix Dump” y seleccionamos nuestro dumpeado, pues cuando estemos en el OEP y con la IAT semi-correcta a causa del call semi¨magico deberemos volcar el artchivo con LordPE por ejemplo que va muy bien, activamos la opcion IntelliDump y hacemos un “dump full” obtendremos el archivo dumpeado en el mismo directorio de instalacion del programa, este es el que seleccionaremos con ImpRec cuando pulsemos “Fix Dump” si miramos sus secciones despues veremos alli una nueva seccion al final llamada .mackt y que ocupa 3000 bytes (en hexa) como vemos en la foto:



Name	Virtual Size	Virtual Offset	Raw Size	Raw Offset	Characteristics
	00076000	00001000	00076000	00001000	C0000040
	00022000	00077000	00022000	00077000	C0000040
	000F6000	00099000	000F6000	00099000	C0000040
.rsrc	0006C000	0018F000	0006C000	0018F000	C0000040
.data	00017000	001FB000	00017000	001FB000	C0000040
.adata	00001000	00212000	00001000	00212000	C0000040

Alli tenemos nuestra nueva IT (Import Table), pues la IAT (Import Address Table) suele estar en .idata, bien ahora que tenemos la IAT resuelta nos queda repara el tema de los stolen bytes, cargamos el archivo en Olly debug y estaremos en el OEP, aquí cambiamos los 00′s que hay por nuestros stolen que hemos pillado ya antes, despues seleccionamos los bytes que hayan sido modificados y con el boton derecho picamos sobre cualquiera de ellos, elegimos “Copy to executable => Selection” se abrira otra ventanita donde con el boton derecho otra vez elegimos “Save file” le damos un nombre, lo abrimos con Peditor y le ponemos el nuevo OEP :42b808 y probamos de ejecutarlo…FUNCIONA…!!!

ASPriotect ha caido una vez mas y la verdad que no ha sido muy dificil, mas bien facildiria y todo, pues es una proteccion bien conocida por todos nosotros y es dificil que nos engañe, jeje

Si probamos a ejecutarlo veremos este cartel:

resulta que antes de desempaquetarlo en mi maquina ya estaba expirado pero podria ser una comprobacion por si lo hemos descomprimido y no arranque, en cualquier caso procederemos a repararlo y ya esta, vamos a ir traceando hasta ver donde lo saca e intentaremos evitarlo.

El mensaje sale aquí:

```
:004227B7    E8 02A80100     CALL <JMP.&mfc42.#2514>
```

Si miramos por encima vemos esto:

```
:00422785    E8 92A70100        CALL <JMP.&mfc42.#800>
:0042278A    84DB               TEST BL,BL
:0042278C    5B                 POP EBX
:0042278D    0F84 FA000000      JE dumped_1.0042288D    ;aqui debera saltar siempre
```

una llamada a la librería MFC42.DLL y una comprobacion posterior de la parte baja de ebx (bl), si forzamos el salto a jmp resulta que no solo dejamos de ver el cartel de expired sino que el programa carga, jeje…

Vemos en la barra de título un feo unregistered al igual que en la ventanita about, además despues muestra otra nag, joder, ahora intenta poner el bitrate a 192 Kbps…

O sea que no se nos permite en versión Trial, esto hay que repararlo **;DDD**

Carguemos el dumpeado en Olly, busquemos la cadena, doble clic sobre ella y caemos aquí:

```
:00421C5A    E8 0DBA0100   CALL <JMP.&mfc42.#2764>
:00421C5F    83F8 FF       CMP EAX,-1
:00421C62    74 47         JE SHORT dumped_1.00421CAB
:00421C64    8A46 6C       MOV AL,BYTE PTR DS:[ESI+6C]
:00421C67    84C0          TEST AL,AL
:00421C69    75 40         JNZ SHORT dumped_1.00421CAB
:00421C6B    6A 00         PUSH 0
:00421C6D    6A 00         PUSH 0
:00421C6F    68 04BD4A00   PUSH dumped_1.004ABD04      ; ASCII "192kbps allowed only in registred version!"
```

Ahi la tienen, pero centremonos en la verificacion de [esi+6c] pues en caso de tener un 1 saltara y no veremos el mensaje, pongamos un BPX en esta instruccion y forcemos la aparicion del mensaje para intentar caer justo encima, pero vemos el mensaje y no paró el Olly, nada buscamos y vemos que se repite la misma secuencia pero en la dirección :41f0ad en este caso el flag es [esi+180] con el boton derecho del raton y seleccionando la opcion "Follow in Dump => Memory Address" vemos que equivale a [3A9120] y que el byte bajo es cero que es el que se esta verificando, a bote pronto se me ocurre modificar la zona de esta manera para ponerle un 1 en dicho byte y saltar por encima del error a ver que pasa, cambio a:

```
:0041F0B7    C686 80010000 0>   MOV BYTE PTR DS:[ESI+180],1
:0041F0BE    90                 NOP
:0041F0BF    EB 40              JMP SHORT dumped_n.0041F101
```

Ahora si puedo seleccionar 192 Kbps, pero quedan mas restricciones, mirando vemos esto:

```
00404AB8    68 50A84900    push 0049A850    ; ASCII "Batch process with more then 5 tasks in registred version only."
```

Aparece tambien aparece en :426461, pongo un BPX en ambos sitios y el que salta es el de

Si intento abrir la ventana IDTag veo que el progrma se cierra automaticamente, podria ser a causa de esto:

```
:0040BF2F    FF15 E87C4700   CALL DWORD PTR DS:[477CE8]
:040BF2F     FF15 E87C4700   CALL DWORD PTR DS:[477CE8]      ; ole32.CoUninitialize
```

pues son imagenes del dumpeado y original, pues el dumpeado no apunta a la API como deberia quiza por eso se cierra por error en la ejecucion, para que apunte a esa API debemos darle su direccion que averiguamos con el CoomandBar poniendo "?CoUninitialize" y nos dice :630e17 que es la verdadera direccion y no la que tiene en la entrada que es :77190e17, ahora debemos desviar el EntryPoint del programa para realizar el injerto:

```
mov dword ptr [477ce8], 630e17
```

o bien buscar un sitio libre donde poder poner la instrucción que son 10 bytes, yo lo hice desviando el EntryPoint hacia :612000 que es el inicio de la sección .adata y alli pongo:

```
mov dword ptr [477ce8], 630e17
push ebp
mov ebp, esp
push -1
jmp 42b80d
```

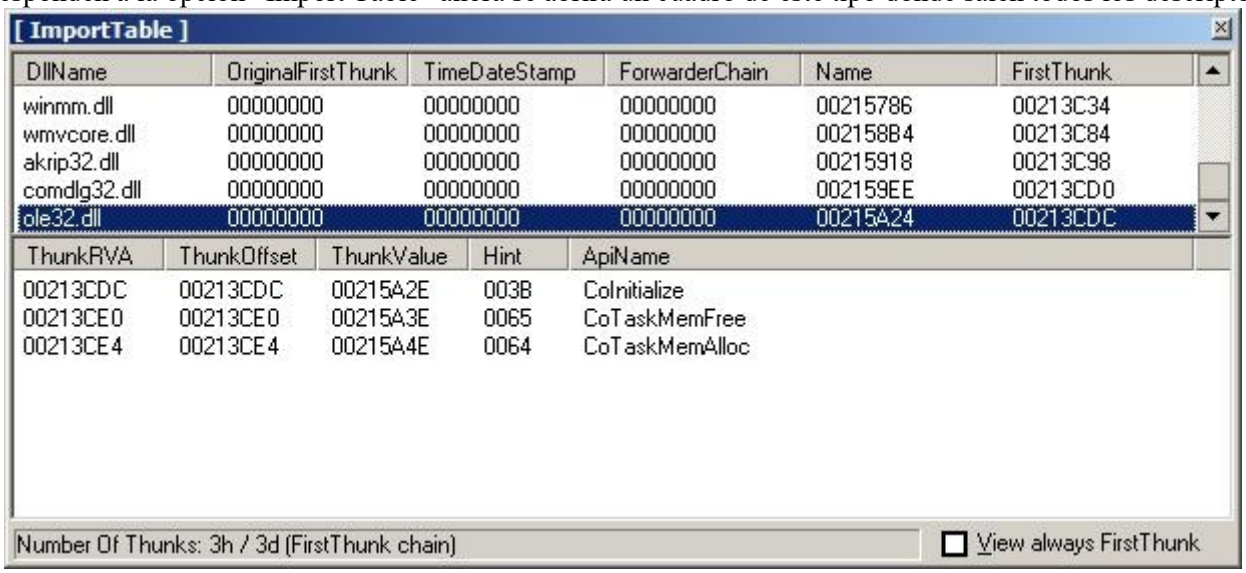
recordad de ejecutar las instrucciones que reemplacemos en el EP y luego saltar allí a :42b80d que es justo después del aslto para no entrar en un bucle tonto, si hacemos esto quedará asi:

```
00612000    C705 E87C4700 1>    MOV DWORD PTR DS:[477CE8],ole32.CoUninitialize
0061200A    55                  PUSH EBP
0061200B    8BEC                MOV EBP,ESP
0061200D    6A FF               PUSH -1
0061200F  - E9 F997E1FF         JMP dumped_n.0042B80D
00612014    90                  NOP
```

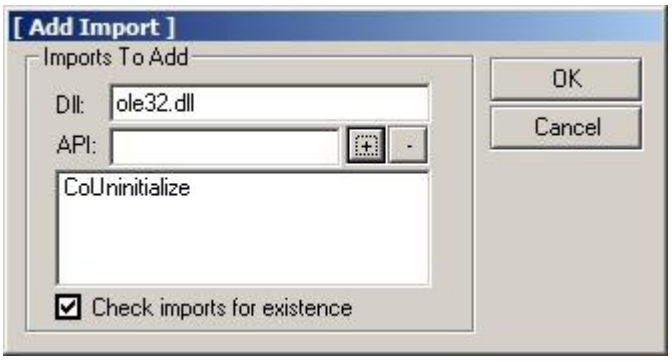
Ahora salvamos los cambios y ejecutamos el nuevo ejecutable y pasa lo mismo, si vamos ahora a la direccion del salto veremos esto:

```
:0040BF2F    FF15 E87C4700          CALL DWORD PTR DS:[477CE8]          ; ole32.00630E71
```

parece que sigue sin apuntar a la API, asçi que va al OLE pero no a donde esto no es normal pues deberia ya saber a don de va a apuntar, ante estos casos lo mejor es utilizar LordPE por ejemplo (vale Peditor, etc...) el programa es lo de menos, lo importante es lo que os voy a explicar, últimamente vengo detectando que muchos programas me borran un descriptor de las funciones importadas de manera que no puede invocarlas desde el proceso, el motivo no lo tengo claro todavía yo trabajo bajo WinXP y antes no me pasaba, siempre que me ocurre es precisamente con funciones del Ole, Oleaut, y funciones de este tipo, no pasandome nunca con las funciones del Kernel, User ni ninguna de estas, ya os digo el motivo lo desconozco y si alguien lo sabe agradecería que me lo dijera porque ya me ha pasado varias veces sobretodo con los Armadillos y ASProtect, bien esto no se ya os digo si es o no normal pero bueno os digo el remedio que se le puede poner si se diera el caso de que a vosotros también os pasara, abrimos LordPE y elegimos la opción “PE Editor” abrimos el archivo dumpeado que da problemas, luego elegimos la opción “Directories” y dentro de esta pulsamos en el botoncillo de los puntos suspensivos que corresponden a la opción “Import Table” ahora se abrirá un cuadro de este tipo donde salen todos los descriptores:



Podemos ver que tengo seleccionada la del ole32.dll que es la que nos está dando problemas, además si miramos no vemos “CoUninitialize” por eso no podemos utilizarla desde el programa, para poder hacerlo debemos añadirla, así en la parte alta de la ventana y con el botón derecho del ratón elegimos “add import…” y se abrirá un pequeño cuadro donde deberemos poner cual queremos añadir, os pongo la images:



Ahora aceptamos y vamos cerrando las ventanas (excepto la del programa pricipal) yo las voy cerrando pulsando "Save" y luego "Ok" despuñes abrimos el nuevo ejecutable y vamos a la entrada a ver si la reconoce ahora:

```
:0040BF2F    FF15 E87C4700   CALL DWORD PTR DS:[477CE8]          ; ole32.CoUninitialize
```

pues si que sale, decir además creo que antes no la reconocía por un error que cometia, pues al darle la dirección le daba 630e71 y no 630e17, pero bueno igualmente hemos aprendido a añadir importaciones que no está nada mal pienso yo, arreglado este tema ahora sí ejecuta la función y no se cierra, jeje así que vamos a por las limitaciones, primero vamos a por la que dice que sólo hace 5 no se qué, ya descubrí como hacer aparecer el mensaje que es este:

Se hace seleccionando más de 5 archivos de música *.wma y con el boton derecho picar en "Add to Batch Queue" después del cartel saca la misma nagscreen de siempre y que ya mismo nos la vamos a cepillar pues se lo merece por pesada ya que sale a cada paso que queremos dar, bueno el cartel sale en :426468 y la nag en :426478

Si miramos el código desde IDA Pro por ejemplo veremos esto, jejeje:

```
:004261ED    call   ebx
:004261EF    mov    cl, [edi+15F8h]      ;bonito flag...!!!
:004261F5    mov    [edi+15A4h], eax
:004261FB    test   cl, cl
:004261FD    jnz    short loc_426208     ;si cl=1 evitamos todo. Ummmhh…!!
:004261FF    cmp    eax, 5               ;hay 5 archivos seleccionados?
:00426202    jge    loc_42645D           ;si hay más de 5 vamos al mensaje restrictor…
```

bueno tenemos ante nuestros ojos la zona donde se decide si ir al mensaje restrictor o por el contrario dejarnos libremente, vamos al mensaje si eax>5, o sea si hey más de 5 archivos seleccionados, pero que hay justo antes? Vemos que cl toma el valor del flag [edi+15f8] y si es distinto de 0 nisiquiera compara ¿quiza prque estemos regisrtados? Esto abre una nueva línea de investigación, iremos a por el flag, vemos que representa a [3ad178] y que además esconde un 0, ¿y si tuviera un 1 que? Pongamoslo a ver que ocurre, pues ocurre que selecciono más de 5 archivos y el programa no se queja ;DDD nos vamos acercando, ya sabemos que si el flag vale 1 es que estamos registrados, reiniciemos y pongamos un "Breakpoint Hardware on Write -> Byte" a ver quien escribe en él, para varias veces pero no importan luego parará aquí:

```
:00423DEA    889E F8150000   MOV BYTE PTR DS:[ESI+15F8],BL
```

en este momento bl vale 0, pongamos un 1 en el flag y puslsemos f9 ya no para más y encima siguen presentes las limitaciones, o sea que no es el flag de registro pero bueno ya podemos utilizar los 192 Kbps y más de 5 archivos, vamos mejorando, ahora centrémonos por esta parte:

```
:00423B57    68 ECE54A00     PUSH dumped_n.004AE5EC          ; ASCII "Unregistered version. %d days left."
```

Encima vemos la comparación:

```
:00423B3C    8A86 AC1B0000     MOV AL,BYTE PTR DS:[ESI+1BAC]
:00423B42    84C0              TEST AL,AL
:00423B44    75 42             JNZ SHORT dumped_n.00423B88
```

donde [esi+1bac] representa a [3a74a4] y vale 0, BPHardware on Write -> Byte en él y caemos aquí:

```
:0042371A    C703 10270000   MOV DWORD PTR DS:[EBX],2710
```

estamos dentro de una rutina que comienza en :423590 e inicializa muchos más flags, este en concreto sólo se encarga de la aparición del Unregistered de la barra de título, o sea cambiemos la táctica por enésima vez, ahora atacaremos por la nag que sale aquí:

```
:0041ABF7   68 A8A04A00     PUSH dumped_n.004AA0A8        ; ASCII "You deserve the best! Purchase"
:0041ABFC   52              PUSH EDX
:0041ABFD   E8 60280200     CALL <JMP.&mfc42.#926>
:0041AC02   68 80A04A00     PUSH dumped_n.004AA080        ; ASCII " to remove all trial restrictions."
```

El retorno nos lleva a una rutina tras pasar por MFC42.DLL que está estremedio, esta rutina empieza en :422c60 y termina en :422e3c y es la encargada de pintar la nag en pantalla, pero tiene un salto condicional que de cumplirse saltará por encima y no la mostrará, la zona está algo ofuscada pero si ponemos un BPX al inicio y la traceamos llegamos a ver esto:

```
:00422CB8   8A86 C4000000   MOV AL,BYTE PTR DS:[ESI+C4]   ;bonito flag!!
:00422CBE   BB EC834700     MOV EBX,dumped_n.004783EC
:00422CC3   84C0            TEST AL,AL                   ;es al=0? Si es 0 algo va mal!!
:00422CC5   BF D8834700     MOV EDI,dumped_n.004783D8
:00422CCA   75 76           JNZ SHORT dumped_n.00422D42  ;salta por encima de la nag...!!!
```

Vemos que el salto depende del valor de [esi+c4] que representa a [58a40c] y en este momento vale 0 con lo que no saltará y veremos la nagscreen, pero ya tenemos otro falg, jeje pongamos un “HBPX on Write Byte” en él y reiniciamos a ver quien escribe en él pero no salta, esto quiere decir que nadie lo modifica, perfecto, recordáis el injerto que pusimos? Le añadiremos la línea:

```
Mov byte ptr [58A40c], 01
```

Este flag es el encargado de controlar la aparición de la nag y poco más, no hemos localizado el flag maestro pero si los auxiliares que modificándolos obtendremos los mismos resultados

Y si hemos seguido todos los pasos hasta aquí tenéis que lo único que queda es quitar el Unregistered de la ventanita About que se puede quitar con cualquier editor de recursos tipo Resource Hacker y cambiarlo por nuestro nombre, el resto es historia.

Y aquí doy por finalizado este tutorial más largo de la cuenta que siempre me pasa, me lio y me lio y acabo escribiendo mogollón pero bueno, otro día nos vemos y seguimos con otras protecciones que serán mejores eguro. Hasta otra y a continuar practicando, jeje

Como siempre para cualquier duda, sugerencia, consulta o crítica podéis escribirme a:

Syxe00@yahoo.es
Syxe00@hotmail.com

Responderé a todo el que escriba, eso seguro.